

Queue Implementation in C++ (With Comments)

Course Notes for Students

1. Introduction to Queue

A Queue is a linear data structure that follows the FIFO principle (First In, First Out). Elements are inserted from the rear end and removed from the front end.

2. Basic Queue Operations

- Enqueue: Insert an element into the queue
- Dequeue: Remove an element from the queue
- Peek: View the front element
- isEmpty: Check if the queue is empty
- isFull: Check if the queue is full

3. Queue Using Array (Linear Queue)

```
class Queue {
    int arr[100];
    int front, rear;

public:
    Queue() {
        front = rear = -1;
    }

    bool isEmpty() {
        return (front == -1 || front > rear);
    }

    bool isFull() {
        return (rear == 99);
    }

    void enqueue(int value) {
        if (isFull()) return;
        if (front == -1) front = 0;
        arr[++rear] = value;
    }

    void dequeue() {
        if (isEmpty()) return;
        front++;
    }
};
```

4. Circular Queue

Circular Queue removes the limitation of unused space by connecting the last position of the array back to the first position.

```
class CircularQueue {
    int arr[100];
    int front, rear, size;

public:
    CircularQueue() {
        front = rear = -1;
    }
};
```

```

        size = 100;
    }

    bool isFull() {
        return ((rear + 1) % size == front);
    }

    bool isEmpty() {
        return (front == -1);
    }

    void enqueue(int value) {
        if (isFull()) return;
        if (front == -1) front = 0;
        rear = (rear + 1) % size;
        arr[rear] = value;
    }

    void dequeue() {
        if (isEmpty()) return;
        if (front == rear)
            front = rear = -1;
        else
            front = (front + 1) % size;
    }
};

```

5. Queue Using Linked List

```

struct Node {
    int data;
    Node* next;
};

class Queue {
    Node* front;
    Node* rear;

public:
    Queue() {
        front = rear = nullptr;
    }

    void enqueue(int value) {
        Node* temp = new Node();
        temp->data = value;
        temp->next = nullptr;

        if (rear == nullptr) {
            front = rear = temp;
        } else {
            rear->next = temp;
            rear = temp;
        }
    }

    void dequeue() {
        if (front == nullptr) return;
        Node* temp = front;
        front = front->next;
        if (front == nullptr) rear = nullptr;
        delete temp;
    }
};

```

```
};
```

6. Summary

- Queue follows FIFO principle
- Linear Queue is simple but inefficient
- Circular Queue optimizes memory usage
- Linked List Queue allows dynamic size